第四次基础实验 实验文档

Last Upd: 2026-10-20

- 第四次基础实验 实验文档
  - 01 实验目标与任务
    - 1 有理数类和封装
    - 2 将类封装为包
    - 3 图神经网络的推理
  - 02 验收内容

01 实验目标与任务

1 有理数类和封装

该实验计划考察类的定义、初始化函数的编写、类方法函数的编写、多态。

将上一个基础实验中实现的有理数表示与运算封装成一个类。参考代码已经给出了类的框架，并给出了相应的测试代码。要求补全其中的重写运算方法，使得 `Rational` 类成为有自己运算与读取方式的一个类型。

参考代码如下

```
# rational_def.py

def gcd(a, b):
    while a != b:
        if a > b:
            a -= b
        else:
            b -= a
    return a

class Rational:
    def __init__(self, n=0, d=1):
        _nu = n; _de = d
        self.__dict__['nu'] = _nu; self.__dict__['de'] = _de

    def __setattr__(self, name, value):
        raise TypeError('Error: Rational objects are immutable')

    def __str__(self): return '%d/%d' % (self.nu, self.de)

    def __add__(self, other):  # addition

    def __sub__(self, other):  # subtraction

    def __mul__(self, other):  # multiplication

    def __truediv__(self, other):  # true division
```

```
    def __eq__(self, other):  # equal

    def __ne__(self, other):  # not equal

    def __gt__(self, other):  # greater than

    def __lt__(self, other):  # less than

    def __ge__(self, other):  # greater or equal

    def __le__(self, other):  # less or equal


def test():
    testsuite = [
        ('Rational(2, 3) + Rational(-70, 40)',
          Rational(-13, 12)),
        ('Rational(-20, 3) - Rational(120, 470)',
          Rational(-976,141)),
        ('Rational(-6, 19) * Rational(-114, 18)',
          Rational(2, 1)),
        ('Rational(-6, 19) / Rational(-114, -28)',
          Rational(-28,361)),
        ('Rational(-6, 19) == Rational(-14, 41)', False),
        ('Rational(-6, 19) != Rational(-14, 41)', True),
        ('Rational(6, -19) > Rational(14, -41)', True),
        ('Rational(-6, 19) < Rational(-14, 41)', False),
        ('Rational(-6, 19) >= Rational(-14, 41)', True),
        ('Rational(6, -19) <= Rational(14, -41)', False),
        ('Rational(-15, 8) == Rational(120, -64)', True),
    ]
    for t in testsuite:
        try:
            result = eval(t[0])
        except:
            print('Error in evaluating ' + t[0]); continue
        if result != t[1]:
            print('Error:  %s != %s' % (t[0], t[1]))
if __name__ == '__main__':
    test()
```

2 将类封装为包

本实验计划考察类的实例化、包封装与调用。

要求将实验二中定义的有理数类封装为包，命名为 `mathtype`，并在和包的同目录（不是包内部）中创建测试脚本 `test_rational.py`，要求其通过 `Rational` 类实例化有理数对象，并计算下式的结果

$$
\begin{aligned}
&666/999 + \frac{233}{114} * \frac{233}{514} - 0.787878 \\
&= \frac{5898312589}{7324500000} \\
&= \text{Rational}(5898312589, 7324500000)
\end{aligned}
$$

```python
# test_rational.py

# Start of TODO 1
# TODO: 从 rational_module 中导入 Rational 类

# End of TODO 1

def test_rational():
    # Start of TODO 2
    # TODO: 通过类实例化定义计算数并进行计算，参照实验二

    # End of TODO 2

    solution = Rational(5898312589, 7324500000)
    print(f"> Your RESULT is: {result}")
    print(f"> The SOLUTION is: {solution}")
    if (result == solution):
        print("Correct! Congratulations! ^o^")
    else:
        print("Wrong... Try again! qwq")

if __name__ == "__main__":
    test_rational()
```

3 图神经网络的推理

在实际的物理系统中，不同状态变量之间往往存在复杂的相互作用。例如，工业设备运行时的载荷变化可能引起温度升高，而温度变化又可能进一步影响设备的振动状态。这些变量不仅具有各自的变化规律，还会通过相互之间的耦合作用，共同决定整个系统的演化过程。因此，要描述一个物理系统的运行状态，除了记录各变量在某一时刻的数值，还需要考虑变量之间的联系及其随时间变化的规律。

图结构为描述这类复杂关系提供了一种自然的方法。在图模型中，可以将系统中的各个状态变量表示为节点，将变量之间的相互作用表示为边，并利用边的权重刻画不同变量之间的影响程度。图神经网络（Graph Neural Network，GNN）进一步引入了消息传递机制，使每个节点能够接收并聚合其他节点的信息，从而更新自身的状态表示。当图结构与动力系统相结合时，便可以同时描述变量之间的关联关系以及系统随时间演化的过程。这一思想也被应用于工业设备的状态建模、故障诊断和剩余使用寿命预测等任务。

本实验构建了一个简化的物理状态演化模型，模型以载荷、温度和振动作为图节点，通过预先设定的连接权重模拟变量之间的相互影响，并利用离散时间步进方法计算系统的状态变化。**实验不涉及神经网络训练**，只保留了图消息传递和动力学演化的基本机制。

对于图中的任意节点 i，首先计算其邻居节点 j 向其传递的消息：

$$m_{j\to i}(t) = w_{ji} x_j(t)$$

其中，$x_j(t)$ 表示节点 j 在时刻 t 的状态，w_{ji} 表示节点 j 对节点 i 的影响权重。随后，节点 i 将所有输入消息进行求和，得到来自其他变量的总影响：

$$a_i(t) = \sum_{j\to i} m_{j\to i}(t) = \sum_{j\to i} w_{ji} x_j(t)$$

这两个过程分别对应图神经网络中的消息计算（Message）和消息聚合（Aggregation）。图神经网络的连接权重已经事先固定，无需执行训练，以简化计算过程。

本实验中，每个节点的变化率由自身的动力学规律及其他节点的影响共同决定：

$$\frac{dx_i}{dt} = b_i - \gamma_i x_i(t) + a_i(t)$$

其中，b_i 为节点自身的基础变化率，γ_i 为阻尼系数，$-\gamma_i x_i(t)$ 表示抑制状态持续增长的作用，而 $a_i(t)$ 则反映其他节点通过图结构产生的影响。通过改变边的连接方式、连接权重或阻尼系数，可以得到不同的系统演化行为。

采用欧拉法对上述连续时间动力学方程进行离散化：

$$x_i(t+\Delta t) = x_i(t) + \Delta t \left[b_i - \gamma_i x_i(t) + \sum_{j \to i} w_{ji} x_j(t) \right]$$

程序每执行一次状态更新，就根据当前时刻各节点的状态计算下一时刻的状态。需要注意的是，所有节点的变化率都应根据同一时刻的状态计算，再统一完成更新。

项目代码已提供，结构如下

```
physics_graph_lab/
├── graph.py           # 待补全：图结构与节点关系
├── dynamics.py        # 待补全：动力学模型、继承与方法重写
├── visualization.py   # 已完成：Matplotlib 实时动画
├── main.py            # 已完成：主程序入口
└── requirements.txt   # 需要安装的包
```

其中， `graph.py` 定义了 `StateGraph` 类，用于管理系统中状态变量之间的有向连接。图中的节点使用字符串标识，例如 "load"、"temperature" 和 "vibration"。每条边表示为 `(source, target, weight)` ，分别对应源节点名称、目标节点名称和连接权重。节点的数值没有保存在图结构中，由动力学模型的 `state` 字典统一管理。

`dynamics.py` 定义了系统动力学和图神经网络的架构与推理。其定义了三个具有继承关系的类。其中， `DynamicsModel` 是动力学模型的基类，提供状态记录、单步更新和多步仿真等通用方法； `IndependentDynamics` 继承基类，实现不考虑节点间影响的动力学计算； `GraphDynamics` 进一步继承 `IndependentDynamics` ，利用图消息传递机制计算节点之间的耦合影响。

`main.py` 提供了固定的网络权重与简单的构建代码， `visualization.py` 则调用 `matplotlib` 库进行推理可视化。

补全代码后，先创建虚拟环境并从 `requirements.txt` 中安装相应的包

```
uv init
uv python pin 3.13
uv sync
uv add -r requirements.txt
```

然后执行 `main.py`

```
uv run main.py
```

也可以用以下一句命令跳过虚拟环境的创建，直接执行

```
uv run --with-requirements requirements.txt python main.py
```

02 验收内容

对于实验 1，其能够正确构建类并跑通给定代码中的测试。

对于实验 2，其运算结果正确。

对于实验 3，其运行能正确可视化推理过程。